Case Study: E-Commerce Order Processing System

Business Background

An e-commerce company named **ShopEasy** is developing an online order processing module.

When a customer places an order:

1. The system validates the order amount.
2. It sends the payment request to an external **Payment Gateway**.
3. If the payment is successful:
    - The order is confirmed.
4. If the payment fails:
    - The system throws an error.
5. If the order amount is invalid (≤ 0):
    - The system should not contact the payment gateway.
    - It should immediately throw an exception.

Since the **Payment Gateway is an external third-party service**, it must NOT be invoked during unit testing.

To ensure proper testing of business logic, the development team has decided to use **Mockito** to mock the payment system.

Technical Implementation

The development team has created the following components:

External Dependency

```
public interface PaymentGateway {
    boolean processPayment(double amount);
}
```

Business Logic Class

```
public class OrderService {

    private PaymentGateway paymentGateway;

    public OrderService(PaymentGateway
paymentGateway) {
        this.paymentGateway = paymentGateway;
    }

    public String placeOrder(double amount) {

        if (amount <= 0) {
            throw new
IllegalArgumentException("Invalid Order Amount");
        }

        boolean paymentStatus =
paymentGateway.processPayment(amount);

        if (!paymentStatus) {
```

```
            throw new RuntimeException("Payment
Failed");
        }

        return "Order Confirmed";
    }
}
```

Your Role as a Backend Developer

You are responsible for writing **unit tests** for `OrderService` using:

- JUnit 5
- Mockito

You must ensure:

- Business logic is tested properly.
- The real `PaymentGateway` is never called.
- All possible scenarios are validated.

Requirements

Create a test class named:

`OrderServiceTest`

Use:

- `@ExtendWith(MockitoExtension.class)`
- `@Mock`
- `@InjectMocks`

Scenario 1: Successful Payment

When:

- Order amount = 2000
- Payment gateway returns `true`

Then:

- Method should return `"Order Confirmed"`
- `processPayment()` must be called once

Scenario 2: Payment Failure

When:

- Order amount = 1500
- Payment gateway returns `false`

Then:

- A `RuntimeException` must be thrown
- `processPayment()` must be called once

Scenario 3: Invalid Order Amount

When:

- Order amount = 0

Then:

- `IllegalArgumentException` must be thrown
- `processPayment()` must NEVER be called

Constraints

- Do not modify the given classes.
- Do not create real implementations of `PaymentGateway`.
- Use Mockito stubbing.
- Verify interactions using `verify()`.